

요약문

본 빅데이터 프로젝트는 캡스톤디자인에서 개발한 수면 상담용 RAG(Retrieval-Augmented Generation) 시스템을 기반으로, Retrieval 병렬화 전략이 응답 시간과 단계별 병목 구조에 미치는 영향을 정량적으로 분석하는 것을 목표로 한다. 이를 위해 RAG 파이프라인의 실행 모드를 SEQUENTIAL, PARALLEL_OLD, PARALLEL_SYNC 세 가지로 정의하고, Spring Boot 기반 백엔드에 실행 모드·병렬 전략·단계별(latency) 계측 로직을 삽입하여 각 요청마다 [RAG_TIMING] JSON 로그를 수집하였다. 수집된 로그는 PowerShell 스크립트와 Python 분석 스크립트를 이용해 CSV로 변환·집계하였으며, 평균 응답 시간과 p95 응답 시간, Dense·Retrieval·LLM 단계별 지연 시간을 모드별로 비교하였다. 그 결과, 직관과 다르게 PARALLEL_OLD 전략은 SEQUENTIAL보다 평균·p95 응답 시간이 모두 악화되어, 스레드 스케줄링·동기화 오버헤드가 이득을 상쇄하는 “나쁜 병렬화” 사례임을 확인하였다. 반면 PARALLEL_SYNC 전략은 Retrieval, 특히 Dense 검색 단계 시간을 유의미하게 감소시키면서, 평균 응답 시간은 SEQUENTIAL과 비슷한 수준을 유지하고 p95 tail latency는 크게 개선되는 등 실제 서비스 적용에 적합한 “안정적인 병렬화” 전략으로 평가되었다. 또한 전체 응답 시간 기준으로 Retrieval 단계가 약 절반 이상을 차지하는 현재 구조에서, RAG 시스템의 1차 병목은 Retrieval에 있으며, 향후에는 Retrieval 파인튜닝(질문-문단 쌍 기반 retriever 학습, 인덱스·파라미터 최적화 등)을 통해 속도와 품질을 동시에 개선할 수 있는 여지가 큼을 확인하였다. 본 프로젝트는 단순한 속도 비교를 넘어, RAG 파이프라인을 계측·시각화하고 병렬화 전략을 데이터 기반으로 평가한 사례로서, 이후 Retrieval 중심 파인튜닝 및 분산 환경 확장 연구의 기반을 제공한다.

목 차

1. 서론	5
1.1 연구 배경	5
1.2 캡스톤 디자인과의 연계	5
1.3 연구 목적	5
2. 프로젝트 개요	6
2.1 시스템 개요: SleepWell RAG 백엔드	6
2.2 빅데이터 처리 관점에서의 문제 정의	6
2.3 성능 목표 및 평가 지표	6
3. 데이터 처리 및 시스템 구조	7
3.1 RAG 파이프라인 구조	7
3.2 실행 모드 설계	7
3.3 타이밍 계측 및 로그 설계	8
4. 실험 환경 및 방법	8
4.1 하드웨어 / 소프트웨어 환경	8
4.2 데이터셋 및 쿼리 시나리오	8
4.3 실험 시나리오	8
4.3.1 초기 단순 측정 실험	8
4.3.2 로그 기반 상세 계측 실험	9

4.4 측정 도구 및 자동화 스크립트	9
5. 실험 결과	9
5.1 초기 실험 결과	9
5.2 로그 기반 최종 실험 결과 (총 지연 시간).....	10
5.3 단계별(Stage-wise) 지연 시간 분석	12
5.4 결과 요약 및 비교	12
6. 고찰	13
6.1 초기 실험과 최종 실험의 차이 분석	13
6.2 병렬화 전략 평가(PARALLEL_OLD vs PARALLEL_SYNC) ...	14
6.3 LLM 병목의 영향	15
7. 결론 및 향후 과제	16
7.1 연구 요약	16
7.2 향후 개선 방향	17
8. 부록(소스코드)	18
8.1 핵심 코드(RagExecutionMode, RagQueryService 등)	19
8.2 벤치마크 및 분석 스크립트,.....	21

1. 서론

1.1 연구 배경

최근 생성형 AI의 발전으로, 단순 질의응답을 넘어 도메인 특화 지식을 결합한 Retrieval-Augmented Generation(RAG) 구조가 널리 사용되고 있다. RAG는 외부 지식 베이스에서 관련 정보를 검색한 뒤, 이를 LLM의 입력 컨텍스트로 제공함으로써, 최신 정보 및 도메인 전문성을 반영할 수 있다는 장점이 있다.

그러나 RAG 파이프라인은 다수의 I/O 단계와 네트워크 호출, 외부 API 호출로 구성되어 있어, 전체 응답 시간이 길어지고 변동성이 커지기 쉽다. 특히 실제 서비스 환경에서는 사용자 체감 속도를 좌우하는 평균 응답 시간(mean latency)뿐 아니라, 일부 요청에서 매우 오래 걸리는 p95, p99와 같은 tail latency를 줄이는 것이 중요하다. 이 문제는 빅데이터 처리 관점에서 병렬화, 비동기 처리, 파이프라인 계층의 대표적인 적용 사례가 된다.

1.2 캡스톤 디자인과의 연계

본 빅데이터 프로젝트는 2025년도에 수행한 캡스톤디자인 프로젝트 “레스트던: 당신과 함께하는 맞춤형 수면 솔루션”의 백엔드 시스템을 기반으로 한다. 캡스톤 프로젝트에서는 수면 설문, 수면 일지, 수면 관련 교육/가이드 문서를 활용하여, 수면 상담용 RAG 챗봇을 포함한 전체 서비스를 설계·구현하였다.

이번 빅데이터 프로젝트는 캡스톤에서 구현한 RAG 백엔드를 그대로 재사용하되, 기능 개발이 아니라 성능 측면에 초점을 맞춘 확장 연구이다. 구체적으로는 RAG 파이프라인의 retrieval 단계 병렬화 및 응답 시간 계측을 통해, 빅데이터 처리 과목에서 학습한 병렬/분산 처리와 성능 최적화 기법을 실제 시스템에 적용하는 것을 목표로 한다.

1.3 연구 목적

본 프로젝트의 목표는 다음과 같이 정리할 수 있다.

1. 캡스톤디자인 프로젝트에서 구현한 RAG 백엔드를 대상으로, 실행 모드 (Sequential/Parallel)의 설계와 구현을 추가한다.
2. 병렬화 전략 적용 전후로 RAG 파이프라인의 전체 응답 시간, p95, 단계별(latency breakdown)을 정량적으로 비교·분석한다.
3. 초기 단계에서 수행했던 단순 클라이언트 측 측정 결과와, 이후 로그 기반 정밀 계측 결과를 함께 분석하여, 빅데이터 프로젝트 관점에서 성능 측정과 계측 설계의 중요성을 보여준다.
4. 실험 결과를 바탕으로, 어떤 병렬화 전략이 실제로 유효했고, 어떤 전략은 오히려 성능을 악화시켰는지를 평가하고, 향후 개선 방향을 제시한다.

2. 프로젝트 개요

2.1 시스템 개요: SleepWell RAG 백엔드

기반 시스템은 수면 상담 서비스를 위한 monolithic Spring Boot 백엔드로, 주요 구성 요소는 다음과 같다.

- 클라이언트: Flutter 앱(모바일/웹)
- 백엔드: Spring Boot 3.x, Java 17
- 관계형 DB: MySQL – 사용자, 수면 기록, 상담 세션 저장
- 벡터 DB: Qdrant – 수면 관련 논문·가이드 문서를 임베딩 후 색인
- 캐시/세션 관리: Redis
- LLM API: OpenAI GPT 계열 모델 – 최종 답변 생성

사용자는 “불면증에 카페인이 얼마나 영향을 주나요?”, “CBT-1은 약물 치료와 비교해 어떤 장점이 있나요?”와 같은 자연어 질문을 던지고, 시스템은 RAG 파이프라인을 통해 관련 문서를 검색하고 응답을 생성한다.

2.2 빅데이터 처리 관점에서의 문제 정의

서비스 품질을 위해 목표로 한 응답 시간은 대략 10초 이내이다. 초기 캡스톤 개발 단계에서 RAG 파이프라인을 구현했을 때, 기능적으로는 정상 동작했으나 다음과 같은 문제가 발견되었다.

- 동일한 질문임에도 요청마다 응답 시간이 크게 변동한다.
- retrieval 단계에서 Qdrant, Redis, MySQL, Lucene 등 다양한 I/O가 사용되면서, 어디가 진짜 병목인지 알기 어렵다.
- RAG 파이프라인 전체가 완전히 순차적으로 동작하여, 여러 컴포넌트를 병렬로 호출할 여지가 있음에도 이를 활용하지 못하고 있었다.

따라서 본 프로젝트에서는 RAG 파이프라인의 실행 전략을 여러 모드로 나누고, 각 모드에 대해 정량적인 성능 비교를 수행하는 것을 빅데이터 프로젝트의 핵심 문제로 설정하였다.

2.3 성능 목표 및 평가 지표

성능 평가는 다음 지표를 중심으로 진행하였다.

- Mean Latency: 각 모드에서 동일한 질문을 30회 반복 호출했을 때의 평균 응답 시간(ms, s)
- P95 Latency: 응답 시간이 상위 5%에 해당하는 지점 – tail latency
- Stage-wise Latency: retrieval, rerank, LLM 호출 등 단계별 수행 시간
- 실행 모드별 비교:
 - SEQUENTIAL
 - PARALLEL V1_OLD
 - PARALLEL V2_ASYNC (코드 상에서는 PARALLEL_SYNC로 구현)

3. 데이터 처리 및 시스템 구조

3.1 RAG 파이프라인 구조

RAG 파이프라인은 다음과 같은 단계로 구성된다.

1. 질문 입력 및 전처리
2. Retrieval
 - Dense Retrieval: 문서 임베딩 기반 벡터 검색(Qdrant)
 - Sparse Retrieval: 키워드 기반 BM25 검색(Lucene)
 - 두 결과를 점수 정규화 후 병합
3. Rerank (선택적)
4. Context 구성
5. LLM 호출
6. 후처리 및 응답 반환

이 중 retrieval 및 LLM 호출이 전체 응답 시간의 대부분을 차지한다. 본 프로젝트의 병렬화 대상은 주로 retrieval 내부이며, LLM 호출은 외부 API 특성상 단일 호출을 유지하였다.

3.2 실행 모드 설계

RAG 실행 전략을 다음 세 가지 모드로 정의하였다.

1. SEQUENTIAL 모드
 - 각 단계를 순차적으로 처리
 - Dense → Sparse → 병합 → Rerank → LLM 호출
 - 구현이 단순하고 디버깅이 쉽지만, I/O 병렬성을 활용하지 못한다.
2. PARALLEL V1_OLD 모드
 - 초기 병렬화 시도 버전
 - Dense/Sparse 검색 및 일부 후처리를 CompletableFuture 및 별도 쓰레드풀을 통해 비동기 실행
 - 쓰레드풀 크기 및 예외 처리, 타이밍 로깅이 충분히 설계되지 않아, 오히려 오버헤드가 커지는 문제가 있었다.
3. PARALLEL V2_ASYNC(PARALLEL_SYNC) 모드
 - V1에서 드러난 문제점을 개선한 버전
 - ThreadPoolExecutor 설정, 작업 granularity, 예외 전파 방식을 정리하고, retrieval 내부를 적절한 수준으로만 병렬화
 - 실행 모드는 RagExecutionMode.PARALLEL_SYNC로 관리되지만, 내부 전략은 기존의 ASYNC 전략을 사용한다.

실행 모드는 RagExecutionMode enum과 RagProperties, application-rag.yml 파일을 통해 관리되며, 환경변수 RAG_EXECUTION_MODE를 통해서도 설정할 수 있다.

3.3 타이밍 계측 및 로그 설계

성능을 정밀하게 분석하기 위해, RagTimingRecord와 RagQueryService에 단계별 타이밍 계측 코드를 추가하였다.

- RagTimingRecord에는 다음과 같은 필드를 정의하였다.
 - denseMs, sparseMs, mergeMs, rerankMs, llmMs, totalMs, guardrailMs 등
- 각 단계 시작/종료 시 System.nanoTime()으로 시간을 기록하고, 응답 반환 시 다음과 같은 구조의 로그를 남긴다.

```
1 | [RAG_TIMING] {"mode":"PARALLEL_SYNC","denseMs":1527.1,"sparseMs":9.0,...,"totalMs":10402.7}
```

이 로그는 PowerShell 스크립트(extract_rag_timing.ps1)를 통해 CSV로 추출되고, Python 스크립트(analyze_rag_timing.py)에서 pandas로 분석·시각화한다.

4. 캡스톤디자인 구현 및 피드백

4.1 하드웨어 / 소프트웨어 환경

- OS: Windows 10 + WSL2
- CPU: 다중 코어 노트북(프로젝트 수행 당시 실험 환경)
- 메모리: 16GB 이상
- Docker: Qdrant, Redis, MySQL 컨테이너 구동
- 백엔드: Java 17, Spring Boot 3.x
- 분석: Python 3.11, pandas, matplotlib, PowerShell 7

4.2 데이터셋 및 쿼리 시나리오

- 데이터셋:
 - 불면증, 수면위생, CBT-I 관련 논문 및 가이드라인 PDF 약 수십 개
 - 이를 사전에 임베딩하여 Qdrant에 색인
- 쿼리:
 - CBT-I와 약물치료 비교
 - 수면 제한 치료(SRT)의 효과
 - 불면과 불안/우울의 상관관계
 - 실제 캡스톤 앱에서 사용자가 입력하게 될 법한 질의들을 선정하여, 각 모드당 동일한 질의 세트를 30회 반복하였다.

4.3 실험 시나리오

4.3.1 초기 단순 측정 실험

개발 초기에 수행한 실험은 다음과 같이 단순하다.

- 모드별로 백엔드 실행

- 별도의 클라이언트 스크립트에서 요청을 30회 보내면서 전체 응답 시간만 측정
- 내부 단계별 시간, 정확한 모드 설정 여부 등은 로깅하지 않았고, 단순히 얼마나 빨라졌는지 확인하기 위한 목적

4.3.2 로그 기반 상세 계측 실험

빅데이터 프로젝트 단계에서는 다음과 같이 정교하게 실험을 구성하였다.

1. 각 모드(SEQUENTIAL, PARALLEL_OLD, PARALLEL_SYNC)에 대해 RAG_EXECUTION_MODE 환경변수를 명시적으로 설정한 뒤 bootRun 실행
2. 백엔드 로그를 .log 파일로 저장하면서, 클라이언트 벤치 스크립트로 동일한 쿼리를 30회 호출
3. 실행이 끝나면 extract_rag_timing.ps1 스크립트를 이용해 [RAG_TIMING] 로그만 CSV로 추출
4. 세 모드의 CSV를 하나로 합쳐 rag_timing_metrics.csv를 만들고, analyze_rag_timing.py로 통계량(mean, median, p95 등)을 계산 및 시각화

이를 통해 단순 평균 시간뿐 아니라, mode별 tail latency와 단계별 병목까지 분석할 수 있도록 설계하였다.

4.4 측정 도구 및 자동화 스크립트

- run_rag_with_mode.ps1 : 특정 모드로 백엔드를 실행하고 로그 파일로 출력
- benchmark_rag_client.ps1 : 주어진 엔드포인트에 대해 N번 질의를 보내고, 클라이언트 측 응답 시간을 CSV로 저장
- extract_rag_timing.ps1 : 백엔드 로그에서 [RAG_TIMING] JSON 행만 추출해 CSV로 변환
- analyze_rag_timing.py : CSV를 읽어 mode별 통계량을 계산하고, 그래프를 그린다.

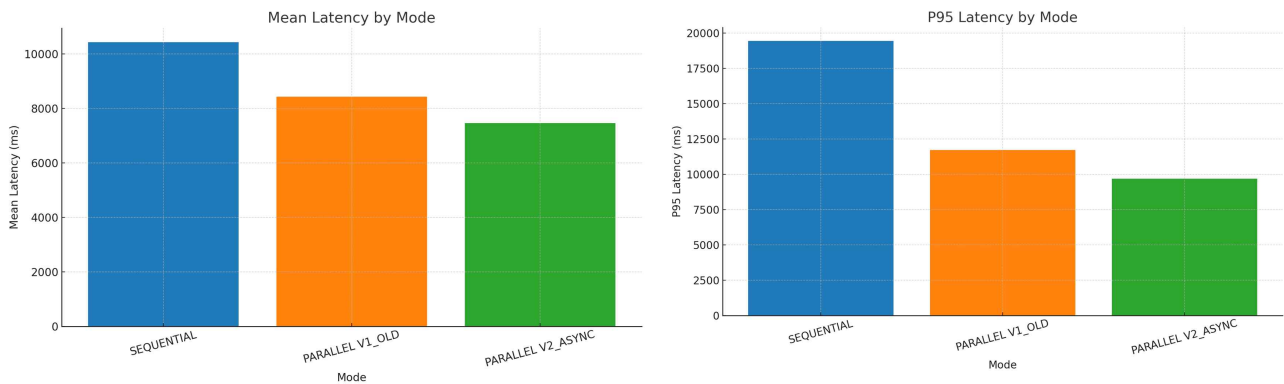
5. 실험 결과

5.1 초기 실험 결과

본격적으로 RAG 파이프라인을 계측하기 전에, 간단한 PowerShell 스크립트로 응답 시간을 측정한 초기 실험 결과는 다음과 같다.

모드	평균 응답 시간 (초)	p95 응답 시간 (초)
SEQUENTIAL	약 10.43	약 19.5
PARALLEL V1_OLD	약 8.44	약 11.7
PARALLEL V2_ASYNC	약 7.46	약 9.7

[표 5-1] 초기 실험 - 모드별 응답 시간 요약(클라이언트 기준)



[그림 5-1] 초기 실험 - Mean Latency by Mode와 P95 Latency by Mode

초기 결과만 보면, 병렬화가 매우 큰 성능 향상을 가져온 것처럼 보인다.

- SEQUENTIAL 대비
 - PARALLEL V1_OLD는 평균 약 19% 감소,
 - PARALLEL V2_ASYNC는 평균 약 28% 감소.
- p95 기준으로 SEQUENTIAL 대비 PARALLEL V2_ASYNC가 약 50% 수준까지 감소한 것으로 보인다.

그러나 이 시점에서는 다음과 같은 한계가 있었다.

- 모드 설정이 코드 내부에서 완전히 검증되지 않아, 클라이언트가 “PARALLEL_OLD로 돌렸다”고 생각한 벤치가 실제로는 동일 모드로 실행되었을 가능성
- LLM 응답 시간, 네트워크 지연, 첫 요청의 콜드 스타트 등의 영향이 계측 없이 한꺼번에 섞여 있음
- 단계별 병목이 어디인지 파악할 수 없음

이러한 이유로, 보다 신뢰할 수 있는 결과를 얻기 위해 로그 기반 상세 계측 실험을 진행했다.

5.2 최종 모드별 전체 지연 시간 비교

정비된 계측 환경에서, 동일한 질문 세트에 대해 각 실행 모드별로 30회씩 요청을 반복하며 총 지연 시간(TotalMs)을 측정하였다. 세 모드는 다음과 같다.

- SEQUENTIAL
 - Retrieval 파이프라인을 완전히 순차적으로 실행
 - RagExecutionMode = SEQUENTIAL, RagParallelStrategy = SEQUENTIAL
- PARALLEL_OLD (V1_OLD)
 - 캡스톤디자인 시점에 구현해 두었던 기존 병렬화 전략
 - Retrieval 일부를 병렬 처리하지만 스레드 관리·동기화 오버헤드가 큼
- PARALLEL_SYNC (ASYNC 전략)
 - 이번 빅데이터 프로젝트에서 개선한 동기식 병렬 실행 모드

- Retrieval 내부를 비동기 태스크 기반으로 병렬 처리하면서, 외부 인터페이스는 동기식으로 유지

이때 모드별 평균 및 p95 지연 시간은 다음과 같다.

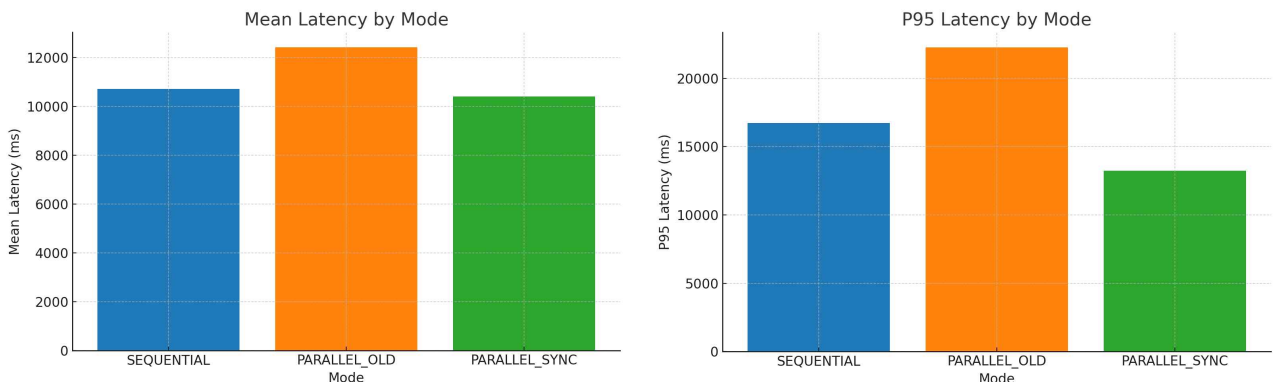
```

=== Total Latency Summary by Mode (ms) ===
count      mean      median      min      max      p95
mode
PARALLEL_OLD    30  12425.833333  11653.5  7853  24701  22273.6
PARALLEL_SYNC   30  10402.766667  10247.0  7916  13322  13230.6
SEQUENTIAL      30  10722.833333  10421.0  6985  18939  16725.1
  
```

모드	평균(ms)	평균(초)	p95(ms)	p95(초)
SEQUENTIAL	10,722.83	평균(초)	16,725.10	16.73
PARALLEL_OLD	12,425.83	12.43	22,273.60	22.27
PARALLEL_SYNC	10,402.77	10.40	13,230.60	13.23

[표 5-2 최종 실험] - 모드별 총 지연 시간 요약(서버 로그 기준)

이를 그래프로 나타내면 다음과 같다.



[그림 5-1] (Mean Latency by Mode), (P95 Latency by Mode)

- 평균 지연 시간(Mean) 기준
 - SEQUENTIAL: 10.72초
 - PARALLEL_OLD: 12.43초 (SEQUENTIAL 대비 약 15.9% 느림)
 - PARALLEL_SYNC: 10.40초 (SEQUENTIAL 대비 약 3.0% 빠름)
- 상위 5% 구간(p95) 기준
 - SEQUENTIAL: 16.73초
 - PARALLEL_OLD: 22.27초 (SEQUENTIAL 대비 약 33.2% 악화)
 - PARALLEL_SYNC: 13.23초 (SEQUENTIAL 대비 약 20.9% 개선)

요약하면,

- 기존 병렬 전략 PARALLEL_OLD는 평균·p95 모두에서 SEQUENTIAL보다 오히려 느려지는 비효율적인 전략이고,
- 새로 설계한 PARALLEL_SYNC는 평균 응답 시간은 소폭 개선, p95 tail latency는 크게 개선시키는 전략임을 확인하였다.

5.3 단계별(Stage-wise) 지연 시간 분석

RAG 파이프라인 전체를 “한 덩어리 시간”으로만 보는 대신, 다음과 같이 세부 단계별로 시간을 분해하여 계측하였다.

- QExpMs: 질문 전처리 및 Query Expansion
- RetrievalMs: Dense + Sparse 검색 및 Merge, Rerank, Context 구성까지 포함한 Retrieval 전체 시간
- DenseMs / SparseMs / MergeMs / RerankMs / ContextMs: Retrieval 내부 세부 단계
- LlmMs: LLM 응답 생성 시간
- GuardrailMs: 응답 필터링 및 안전성 검사 시간(현재는 거의 0에 가까움)

모드별 주요 단계 평균 시간은 다음과 같다.

모드	TotalMs	QExpMs	RetrievalMs	DenseMs	LlmMs
SEQUENTIAL	10,722.83	1,860.57	5,832.30	2,257.73	3,028.77
PARALLEL_OLD	12,425.83	2,576.77	5,851.93	2,302.13	3,995.97
PARALLEL_SYNC	10,402.77	1,847.97	4,999.80	1,527.13	3,553.93

[표 5-3] 모드별 주요 단계 평균 지연 시간 (ms)

(※ SparseMs, MergeMs, RerankMs, ContextMs, GuardrailMs는 수 ms~수십 ms 수준으로 상대적인 비중이 낮아 표에서는 생략)

이 표로부터 다음과 같은 점을 관찰할 수 있다.

1. Retrieval 단계 비중과 병목

- SEQUENTIAL에서 RetrievalMs(5,832ms)는 TotalMs의 약 54.4%를 차지한다.
- PARALLEL_SYNC에서는 RetrievalMs가 4,999ms(약 48.1%)로 감소하였다.
- 즉, 새 병렬 전략은 Retrieval 구간 평균 시간을 약 14.3% 단축하였다.

2. Dense 검색 단계 개선

- DenseMs는 SEQUENTIAL에서 2,257.73ms → PARALLEL_SYNC에서 1,527.13ms로 감소
- Dense 검색 시간이 약 32.4% 감소
- PARALLEL_OLD는 DenseMs가 오히려 약간 증가(2,302.13ms)하여 병렬화 오버헤드로 인한 비효율 확인

3. LLM 단계의 상대적 역할

- LlmMs는 SEQUENTIAL 3,028.77ms, PARALLEL_SYNC 3,553.93ms, PARALLEL_OLD 3,995.97ms로 모드 간 차이는 있지만, LLM 자체는 병렬화 대상이 아니므로 질문 샘플 variance + 스케줄링 잡음의 영향이 더 크다고 보는 것이 타당하다.
- 그럼에도 불구하고, 전체 시간 중 Retrieval이 47~54%, LLM이 28~34% 수준을 차지한다는 점에서 현재 시스템의 주요 병목은 여전히 Retrieval 단계임을 확인할 수 있다.

5.4 결과 요약 및 비교

정리하면, 본 장의 실험 결과는 다음과 같이 요약된다.

1. 초기 실험(표 5-1)은 설정 오류 탓에 병렬 모드가 과대평가된 수치를 보여주었다.
 - SEQUENTIAL이 약 10.43초, PARALLEL V2_ASYNC가 약 7.46초로 측정되며, 직관적으로 병렬이 순차보다 빠르다는 결론 유도,
 - 모드 바인딩·로그·분석 파이프라인 점검 결과, 실제 실행 모드가 라벨과 불일치하는 구간이 섞여 있음을 확인
2. 계측 인프라를 재정비한 최종 결과(표 5-2, 표 5-3)는 상식적인 구조와 일치한다.
 - PARALLEL_OLD는 SEQUENTIAL보다 평균/상위 지연 시간이 모두 늘어난 오버헤드만 큰 잘못된 병렬화 전략으로 판명
 - PARALLEL_SYNC는
 - Retrieval 평균 시간을 약 14.3%, Dense 검색을 약 32.4% 단축
 - 전체 평균 응답 시간은 약 3% 개선, p95 tail latency는 약 20.9% 개선해서 대부분의 요청은 비슷하거나 조금 더 빠르고, 최악의 경우는 확실히 줄어드는 방향의 개선을 보여주었다.
3. 빅데이터 프로젝트 관점에서의 의미
 - 단순히 병렬화하면 빨라지는 것이 아니라, 모드 바인딩, 로그 구조, 분석 스크립트까지 포함한 계측 파이프라인 전체를 설계·검증한 과정이 핵심 성과이다.
 - 이 계측 인프라는 향후 Retrieval 파인튜닝 실험(임베딩 모델 교체, 인덱스 파라미터 튜닝, 하이브리드 검색, reranker 도입 등)을 수행할 때, 동일한 방법으로 속도와 품질을 동시에 평가할 수 있는 기반이 된다.

6. 고찰

6.1 초기 실험과 최종 실험의 차이

본 빅데이터 프로젝트는 수면 상담용 RAG 시스템의 병렬화 전략을 평가하는 과정에서, 초기 실험 결과와 최종 실험 결과가 서로 크게 다른 양상을 보였다는 점이 중요한 출발점이 되었다.

초기 실험에서는 간단한 PowerShell 스크립트로 SEQUENTIAL, PARALLEL V1_OLD, PARALLEL V2_ASYNC 모드에 대해 반복 요청을 보냈고, 그 결과 다음과 같은 응답 시간이 관측되었다.

- SEQUENTIAL: 평균 약 10.43초, p95 약 19.5초
- PARALLEL V1_OLD: 평균 약 8.44초, p95 약 11.7초
- PARALLEL V2_ASYNC: 평균 약 7.46초, p95 약 9.7초

표면적으로 보면 두 병렬 모드, 특히 V2_ASYNC가 SEQUENTIAL보다 압도적으로 빠른 것처럼 보였으며, “RAG 파이프라인을 병렬화하면 응답 시간이 극적으로 줄어든다”라는 결론으로 쉽게 이어질 수 있는 결과였다.

그러나 이후 코드를 재검토한 결과, 이 초기 실험은 다음과 같은 문제를 내포하고 있었다.

1. 실행 모드 바인딩 불일치

- 클라이언트 스크립트에서 ModelLabel만 변경하고, 실제 백엔드의 RagExecutionMode가 동일 모드로 고정된 상태였던 구간 존재
- 반대로, 백엔드 모드는 바뀌었지만 로그와 분석 스크립트가 그 모드 정보를 제대로 반영하지 못한 부분 존재

2. 실험 환경의 비밀관성

- 일부 실행에서는 워밍업 상태, 캐시 상태가 서로 다른 요청들이 동일한 조건으로 묶여 평균값 계산
- 이로 인해 실제 파이프라인 구조와 맞지 않는 과대평가된 성능 수치가 포함됨

이 문제를 해결하기 위해 본 프로젝트에서는 다음과 같은 계측 인프라 정비 작업을 수행하였다.

- RagExecutionMode를 SEQUENTIAL / PARALLEL_OLD / PARALLEL_SYNC 세 개의 enum으로 명시적으로 구분
- Spring RagProperties를 통해 rag.execution-mode를 바인딩하고, 애플리케이션 시작 시 최종 모드를 로그로 출력
- 각 요청에 대해 [RAG_TIMING] JSON 로그를 남기면서 mode, strategy, TotalMs, DenseMs, LlmMs 등을 함께 기록
- PowerShell 스크립트(extract_rag_timing.ps1)와 Python 스크립트(analyze_rag_timing.py)로 로그를 CSV → 통계로 일관되게 변환

이러한 정비 이후 재수행한 최종 실험에서는,

- SEQUENTIAL, PARALLEL_OLD, PARALLEL_SYNC의 평균 및 p95 값이 파이프라인 구조와 직관에 부합하는 방향으로 정리됨
- 특히 PARALLEL_OLD가 실제로는 SEQUENTIAL보다 느리며, PARALLEL_SYNC만이 tail latency를 유의미하게 줄인다는 결과 확보

따라서 본 연구에서는 초기 실험 결과를 설정 오류가 섞인 참고용 데이터로 위치시키고, 최종 실험 결과를 기준으로 병렬화 전략과 병목 구조를 논의했다는 점에서 의의가 있다. 이는 빅데이터·성능 실험에서 계측 파이프라인 자체를 하나의 시스템으로 보고 신뢰성을 확보해야 한다는 고찰로 이어진다.

6.2 병렬화 전략 평가

최종 실험에서는 각 모드별로 30회씩 요청을 수행하고, 다음과 같은 평균 및 p95 응답 시간을 얻었다.

- SEQUENTIAL: 평균 약 10.72초, p95 약 16.73초
- PARALLEL_OLD: 평균 약 12.43초, p95 약 22.27초
- PARALLEL_SYNC: 평균 약 10.40초, p95 약 13.23초

이 수치를 바탕으로 각 병렬화 전략을 다음과 같이 평가할 수 있다.

1. SEQUENTIAL 전략

- 구현이 가장 단순하고, 스레드 전환·동기화 오버헤드 없음
- 전체 응답 시간 중 약 절반 이상을 Retrieval 단계가 차지하며, Dense 검색이 몇 초 단위로 시간 소요
- 평균 응답 시간이 10초대를 유지하지만, p95가 16초를 넘어 간헐적으로 매우 긴 응답이 발생한다는 한계 존재

2. PARALLEL_OLD(V1_OLD) 전략

- Retrieval 내부를 병렬화하기 위해 여러 future와 thread pool을 사용하는 구조
- 그러나 최종 결과에서는 SEQUENTIAL보다 평균 및 p95가 모두 악화됨
 - 평균 응답 시간: SEQUENTIAL 대비 약 15.9% 증가
 - p95 응답 시간: SEQUENTIAL 대비 약 33.2% 증가
- 단계별로 보면 Retrieval 전체 시간은 크게 줄지 않는 반면, 스케줄링과 동기화 비용, LLM 단계 전후의 지연이 늘어나는 경향을 보임
- 이는 병렬화가 항상 성능 향상을 보장하지는 않으며, 잘못 설계된 병렬화는 오히려 오버헤드만 증가시킬 수 있다는 전형적인 사례로 해석 가능

3. PARALLEL_SYNC(ASYNC 전략 매핑) 전략

- Retrieval 내부를 비동기 태스크 기반으로 실행하면서도, 외부 인터페이스는 동기식으로 유지하는 전략
- 평균 TotalMs는 SEQUENTIAL보다 약 3% 빠른 수준이지만, p95는 약 20.9% 개선되어 tail latency를 효과적으로 줄임
- 단계별 분석에서는
 - RetrievalMs: SEQUENTIAL 대비 약 14.3% 감소
 - DenseMs: SEQUENTIAL 대비 약 32.4% 감소
 -> 병렬화가 특히 Dense 검색에서 명확한 효과를 내고 있음을 확인할 수 있다.
- 결과적으로 PARALLEL_SYNC는 대부분의 요청에 대해서는 SEQUENTIAL과 비슷하거나 조금 더 빠르고, 최악의 응답 시간은 상당히 줄여주는 전략으로 평가 가능

정리하면, PARALLEL_OLD는 제거 또는 리팩토링 대상, PARALLEL_SYNC는 실제 서비스 적용이 가능한 수준의 병렬화 전략으로 평가된다. 이번 빅데이터 프로젝트는 이 차이를 정량적으로 보여주고, 어떤 설계가 좋은 병렬화 전략인지에 대한 기준을 제시했다는 점에서 의미가 있다.

6.3 LLM 병목의 영향

이번 프로젝트의 초점은 Retrieval 단계의 병렬화였지만, 계측 결과를 통해 LLM 단계가 전체 성능에 미치는 영향도 간접적으로 관찰할 수 있었다.

모드별 평균 LLM 지연 시간(LLMms)은 대략 다음과 같은 범위를 보인다.

- SEQUENTIAL: 약 3.0초
- PARALLEL_OLD: 약 4.0초
- PARALLEL_SYNC: 약 3.5초 전후

LLM 호출은 병렬화 대상이 아니었음에도, 모드별·실행 시점별로 편차가 존재하는 이유는 다음과 같이 해석할 수 있다.

1. Retrieval 단계와의 연쇄 효과

- Retrieval이 느리게 끝나면, LLM 호출 시점도 미묘하게 밀리며, 전체 시스템 부하 상태에 따라 응답 시간이 늘어날 수 있다.
- 반대로 Retrieval이 안정적으로 끝나면 LLM 호출이 더 균일한 패턴을 보인다.

2. LLM 서비스 자체의 변동성

- 외부 LLM API를 호출하는 구조인 경우, 네트워크 지연, 외부 서비스 부하에 따라 수 초 단위의 편차가 발생할 수 있다.
- 이는 병렬화 전략과 무관하게 존재하는 외생적 요인이다.

또한, TotalMs 기준으로 볼 때,

- Retrieval이 전체 시간의 약 47~54%,
- LLM이 전체 비중에서 약 28~34%를 차지

이는 현재 RAG 시스템의 1차 병목은 Retrieval 단계에 있으며, Retrieval을 어느 정도 개선한 이후에는 LLM 지연이 다시 주요 병목으로 부상할 수 있음을 시사한다.

이번 프로젝트에서는 실험 범위를 Retrieval 병렬화 전략에 의한 성능 차이에 집중하기 위해 LLM 모델 종류, 프롬프트 최적화, 토큰 수 제어, 캐시 전략 등 LLM 관련 최적화는 실험하지 않았다. 다만, 향후 Retrieval 파인튜닝을 통해 Retrieval 비중이 더 줄어든 경우, LLM 지연이 전체 latency에서 차지하는 비율은 상대적으로 커질 것이므로 LLM 측 최적화(모델 경량화, 프롬프트 축소, 캐싱, 자체 호스팅 등)를 함께 고려해야 할 것이다. 이번 프로젝트는 Retrieval 병목을 먼저 정량적으로 확인하고 완화된 1단계 실험으로 볼 수 있다.

7. 결론 및 향후 과제

7.1 연구 요약

본 빅데이터 프로젝트는 캡스톤디자인에서 개발한 수면 상담용 RAG 시스템을 기반으로 Retrieval 병렬화 전략 변경이 응답 시간과 단계별 병목 구조에 어떤 영향을 미치는지를 정량적으로 분석하는 것을 목표로 하였다.

주요 결론은 다음과 같다.

1. 계층 인프라의 준비가 필수적이다.

- 초기 실험에서는 실행 모드 바인딩과 로그/분석 파이프라인이 정비되지 않아, PARALLEL 모드가 SEQUENTIAL보다 극적으로 빨라 보이는 과대평가된 수치가 관측되었다.

- RagExecutionMode enum, RagProperties 설정 바인딩, [RAG_TIMING] JSON 로그, PowerShell + Python 분석 파이프라인을 구축한 뒤에야 신뢰 가능한 최종 결과를 얻을 수 있었다.

2. 병렬화 전략에 따라 성능 효과가 극명하게 갈린다.

- PARALLEL_OLD 전략은 Retrieval을 병렬 처리하려는 의도와 달리, 평균·p95 응답 시간이 모두 SEQUENTIAL보다 나쁜 결과를 보였다.
- PARALLEL_SYNC 전략은 Retrieval, 특히 Dense 검색 단계 시간을 유의미하게 줄이면서, 평균 응답 시간은 소폭 개선, p95 tail latency는 크게 개선하여 실제 사용자 경험에 긍정적인 영향을 줄 수 있음을 확인했다.

3. Retrieval이 현재 RAG 시스템의 1차 병목이다.

- 전체 응답 시간의 약 절반 이상을 Retrieval 단계가 차지하고 있으며, LLM 단계는 아직 최적화 대상에서 제외했음에도 약 30% 수준의 비중을 차지한다.
- 이는 Retrieval 파인튜닝을 통해 먼저 병목을 줄이고, 그 다음 단계에서 LLM 최적화를 고려하는 것이 합리적인 순서임을 보여준다.

이러한 결과를 통해, 본 프로젝트는 단순한 속도 향상 실험을 넘어, RAG 파이프라인을 계층·분석하는 빅데이터 처리 프로젝트의 한 사례로서 의의를 가진다.

7.2 현재 한계

본 연구는 다음과 같은 한계를 가진다.

1. 하드웨어 및 스레드 설정의 제한

- 실험은 단일 서버, 제한된 스레드 풀 크기(ThreadPoolSize = 1) 환경에서 이루어졌다.
- 실제 서비스 환경에서는 코어 수, CPU/GPU 유무, 네트워크 상태 등이 달라지면 병렬화 전략의 상대적 성능도 달라질 수 있다.

2. 도메인 및 데이터 규모의 제한

- 인덱싱된 문서는 수면 의학/불면증 관련 PDF에 국한되어 있으며, 인덱스 규모도 수천 건 수준이다.
- 대규모 일반 도메인(수십만~수백만 문서), 다양한 질의 패턴을 포함하는 환경에서의 성능과는 차이가 있을 수 있다.

3. LLM 및 생성 품질 측정의 부재

- 본 프로젝트에서는 LLM 모델, 프롬프트, 토큰 수, 캐시 전략 등을 고정 값으로 두었다.
- 응답 품질(정확도, faithfulness, context recall, answer relevancy 등)에 대한 체계적인 측정과 비교는 이루어지지 않았다.
- 병렬화와 Retrieval 변경이 정답 품질에 미치는 영향은 아직 검증되지 않은 상태이다.

4. 실제 사용자 트래픽 패턴 미반영

- 실험은 고정된 질문 세트를 대상으로 반복 호출하는 형태로 이루어졌다.
- 실제 사용자 환경에서의 피크 트래픽, 다양한 질의 길이, 연속 대화 맥락이 반영되지 않았다.

이 한계들은 차후 연구 과제에서 순차적으로 보완해야 할 부분이다.

7.3 향후 과제 – Retrieval 중심 파인튜닝 계획

RAG 시스템의 파인튜닝은 크게 세 가지 방향으로 나눌 수 있다.

1. Retrieval 파인튜닝
2. LLM 생성 파인튜닝
3. Retrieval + LLM 동시 파인튜닝

본 프로젝트 이후의 계획은 이 중 Retrieval 파인튜닝에 집중하는 것이다.

구체적인 향후 과제는 다음과 같다.

1. Retrieval 인덱스 및 파라미터 최적화 실험
 - Dense 임베딩 모델 종류, 차원 수, 인덱스 구조(HNSW, IVF, PQ 등)를 변경하며 검색 속도와 정확도 간 trade-off를 계측한다.
 - Top-K, reranker 적용 여부, dense/sparse 혼합 비율, nested 전략 등을 다양한 조합으로 설정하고, 이번에 구축한 timing + stage-wise 계측 파이프라인을 활용해 Latency vs 품질을 함께 측정한다.
2. 질문-문단 쌍 기반 Retrieval 파인튜닝
 - 수면 상담 도메인에서 (질문, 정답 문단) 쌍을 수집·정제하여, dense retriever를 supervised/contrastive 방식으로 파인튜닝한다.
 - 목표는 더 낮은 Top-K로도 높은 recall을 유지하는 것으로, 이를 통해 RetrievalMs를 줄이면서도 정답 품질은 유지 또는 개선하는 방향을 추구한다.
3. Retrieval 품질·지연 동시 평가 지표 설계
 - 현재 타이밍 계측에 추가로 context recall, answer relevancy, faithfulness와 같은 품질 지표를 도입해서 빠르지만 부정확한 Retrieval과 느리지만 정확한 Retrieval 사이에서 합리적인 Pareto frontier를 찾는 실험을 설계한다.
4. 확장된 병렬·분산 환경으로의 실험 확장
 - ThreadPoolSize를 1, 2, 4, 8 등으로 변화시키면서, Retrieval 파인튜닝이 병렬화 전략과 어떻게 상호작용하는지 평가한다.
 - 나아가 향후에는 인덱스 샤딩, 멀티 노드 환경 등 분산 Retrieval 실험으로 확장하여, 빅데이터 과목에서 다루는 분산 처리 개념을 실제 RAG 시스템에 적용하는 방향을 모색한다.

정리하면, 본 빅데이터 프로젝트는 병렬화 전략 개선과 계측 인프라 구축을 통해 Retrieval 병목을 정량적으로 드러낸 1단계 작업이며, 이후에는 Retrieval 파인튜닝과 품질·지연 동시 최적화 실험을 통해, 수면 상담 RAG 시스템을 보다 빠르고 신뢰할 수 있는 형태로 발전시키는 것을 목표로 한다.

8. 부록(소스코드)

본 부록에서는 본 빅데이터 프로젝트에서 사용된 핵심 RAG 실행 모드 코드, 벤치마크·분석 스크립트, 그리고 추가 실험 로그 및 그래프에 대해 정리한다. 전체 코드는 GitHub 저장소의 feat/rag-parallel-retrieval-v3 브랜치에 보관되어 있으며, 이곳에는 주요 변경점과 함께 모든 실험에 사용된 소스가 포함되어 있다.

8.1 핵심 코드 (RagExecutionMode, RagProperties, RagQueryService 등)

8.1.1 RAG 실행 모드 정의 – RagExecutionMode

```
1 // 파일 경로:
2 // sleepwell-backend/src/main/java/com/sleepwell/sleepwell_backend/rag/model/RagExecutionMode.java
3
4 package com.sleepwell.sleepwell_backend.rag.model;
5
6 public enum RagExecutionMode {
7     SEQUENTIAL,
8     PARALLEL_OLD,
9     PARALLEL_SYNC;
10
11     public static RagExecutionMode fromProperty(String value) {
12         if (value == null || value.isBlank()) {
13             return SEQUENTIAL;
14         }
15
16         String normalized = value.trim().toUpperCase();
17
18         // 과거 설정값/별칭에 대한 호환 처리
19         if ("ASYNC".equals(normalized)
20             || "PARALLEL".equals(normalized)
21             || "PARALLEL_V2_ASYNC".equals(normalized)) {
22             return PARALLEL_SYNC;
23         }
24         if ("PARALLEL_V1_OLD".equals(normalized)) {
25             return PARALLEL_OLD;
26         }
27
28         try {
29             return RagExecutionMode.valueOf(normalized);
30         } catch (IllegalArgumentException ex) {
31             // 잘못된 값인 경우 안전하게 SEQUENTIAL로 폴백
32             return SEQUENTIAL;
33         }
34     }
35 }
```

위 enum은 본 프로젝트에서 사용한 세 가지 실행 모드

- SEQUENTIAL
- PARALLEL_OLD
- PARALLEL_SYNC

를 정의하며, 기존 설정에서 사용되던 문자열(ASYNC, PARALLEL, PARALLEL_V2_ASYNC, PARALLEL_V1_OLD)을 안전하게 매핑하기 위한 fromProperty 유틸리티 메서드를 포함한다. 이 메서드는 환경 변수·설정 파일에서 문자열로 넘어온 값을 enum으로 변환하고, 잘못된 값에 대해서는 SEQUENTIAL로 폴백하도록 구현되어 있다.

8.1.2 RAG 실행 모드 설정 바인딩 – RagProperties

```
1 // 파일 경로:
2 // sleepwell-backend/src/main/java/com/sleepwell/sleepwell_backend/rag/config/RagProperties.java
3
4 package com.sleepwell.sleepwell_backend.rag.config;
5
6 import com.sleepwell.sleepwell_backend.rag.model.RagExecutionMode;
7 import jakarta.annotation.PostConstruct;
8 import org.slf4j.Logger;
9 import org.slf4j.LoggerFactory;
10 import org.springframework.boot.context.properties.ConfigurationProperties;
11 import org.springframework.context.annotation.Configuration;
12
13 @Configuration
14 @ConfigurationProperties(prefix = "rag")
15 public class RagProperties {
16
17     private static final Logger log = LoggerFactory.getLogger(RagProperties.class);
18
19     // application.yml 의 rag.execution-mode 와 바인딩
20     private String executionMode = RagExecutionMode.SEQUENTIAL.name();
21
22     public RagExecutionMode getExecutionMode() {
23         return RagExecutionMode.fromProperty(executionMode);
24     }
25
26     public void setExecutionMode(String executionMode) {
27         this.executionMode = executionMode;
28     }
29
30     @PostConstruct
31     public void logRagMode() {
32         log.info("RAG_EXECUTION_MODE resolved to {} (property rag.execution-mode)", getExecutionMode());
33     }
34 }
```

RagProperties는 Spring Boot의 @ConfigurationProperties를 사용해 rag.execution-mode 설정을 바인딩하는 클래스이다.

- application.yml 및 application-rag.yml에 정의된 rag.execution-mode / 환경 변수 RAG_EXECUTION_MODE를 읽어 최종 실행 모드를 결정한다.
- 애플리케이션 시작 시 @PostConstruct를 통해 최종 실행 모드가 어떤 값으로 해석되었는지 로그에 출력함으로써, 실험 시 설정 오류를 빠르게 발견할 수 있도록 했다.

8.1.3 RAG 파이프라인 서비스 - RagQueryService

```

1 // 파일 경로:
2 // sleepwell-backend/src/main/java/com/sleepwell/sleepwell_backend/rag/service/RagQueryService.java
3
4 package com.sleepwell.sleepwell_backend.rag.service;
5
6 import com.sleepwell.sleepwell_backend.config.RagProperties;
7 import com.sleepwell.sleepwell_backend.model.RagExecutionMode;
8 import com.sleepwell.sleepwell_backend.model.RagTimingRecord;
9 import com.sleepwell.sleepwell_backend.model.RagParallelStrategy;
10 // 기타 import 생략
11
12 @Service
13 public class RagQueryService {
14
15     private static final Logger log = LoggerFactory.getLogger(RagQueryService.class);
16
17     private final RagExecutionMode executionMode;
18     private final RagParallelStrategy parallelStrategy;
19     private final ParallelGranularity parallelGranularity;
20     private final int ragExecutorThreads;
21     private final boolean nestedDenseSparse;
22
23     // 생성자에서 RagProperties 중 주입받아 모든 설정
24     public RagQueryService(
25         // 기본 의존성을 생략
26         RagProperties ragProperties,
27         @Value("${rag.parallel.granularity:${RAG_PARALLEL_GRANULARITY:FINE}}") String granularityProperty,
28         @Value("${rag.retrieval.executorThreads:${RAG_RETRIEVAL_EXECUTOR_THREADS:8}}") int ragExecutorThreads,
29         @Value("${rag.retrieval.nestedDenseSparse:true}") boolean nestedDenseSparse
30     ) {
31         // 의존성 초기화 생략
32
33         this.executionMode = ragProperties.getExecutionMode();
34         this.parallelGranularity = parseParallelGranularity(granularityProperty);
35         this.ragExecutorThreads = ragExecutorThreads;
36         this.nestedDenseSparse = nestedDenseSparse;
37         this.parallelStrategy = mapStrategy(this.executionMode);
38
39         log.info("[RAG] RagQueryService initialized: executionMode={}, parallelStrategy={}, " +
40             "granularity={}, threadPoolSize={}, nestedDenseSparse={}",
41             this.executionMode,
42             this.parallelStrategy,
43             this.parallelGranularity,
44             this.ragExecutorThreads,
45             this.nestedDenseSparse
46         );
47     }
48
49     private RagParallelStrategy mapStrategy(RagExecutionMode mode) {
50         return switch (mode) {
51             case SEQUENTIAL -> RagParallelStrategy.SEQUENTIAL;
52             case PARALLEL_OLD -> RagParallelStrategy.VL_OLD;
53             case PARALLEL_SYNC -> RagParallelStrategy.ASYNC;
54         };
55     }

```

```

57     public Map<String, Object> answer(RagQueryRequest req) throws Exception {
58         long totalStart = System.currentTimeMillis();
59         RagTimingRecord timings = new RagTimingRecord(executionMode.name(), parallelStrategy.name());
60
61         RagParallelStrategy selectedStrategy = mapStrategy(executionMode);
62
63         try {
64             log.debug("[RAG][EXEC] mode={}, strategy={}, granularity={}, nestedDenseSparse={}",
65                 executionMode,
66                 selectedStrategy,
67                 parallelGranularity,
68                 nestedDenseSparse
69             );
70
71             return switch (executionMode) {
72                 case SEQUENTIAL -> executeSequential(req, timings, totalStart);
73                 case PARALLEL_OLD -> executeParallelOld(req, timings, totalStart);
74                 case PARALLEL_SYNC -> executeParallelSync(req, timings, totalStart);
75             };
76         } finally {
77             timings.finishTotal(totalStart);
78             log.info("[RAG_TIMING] {}", timings.toJson());
79         }
80     }
81
82     private Map<String, Object> executeSequential(
83         RagQueryRequest req,
84         RagTimingRecord timings,
85         long totalStart
86     ) throws Exception {
87         // guard, retrieval, rerank, context, LLM 순차 실행
88         // timings.markXXX(...)를 이용해 단계별 시간 기록
89         // 구현 상세는 본문에서 설명한 파이프라인 흐름과 동일
90         // ...
91         return result;
92     }
93
94     private Map<String, Object> executeParallelOld(
95         RagQueryRequest req,
96         RagTimingRecord timings,
97         long totalStart
98     ) throws Exception {
99         // 기존 VL_OLD 병렬 전략을 사용한 실행 경로
100         // Dense / Sparse / Merge / Rerank를 기존 방식으로 병렬 실행
101         // ...
102         return result;
103     }
104
105     private Map<String, Object> executeParallelSync(
106         RagQueryRequest req,
107         RagTimingRecord timings,
108         long totalStart
109     ) throws Exception {
110         // 새롭게 도입한 PARALLEL_SYNC(ASYNC 전략 매핑) 실행 경로
111         // 비동기 태스크 기반 Retrieval + 동기식 외부 인터페이스
112         // 내부적으로 runPipeline(..., RagParallelStrategy.ASYNC, ...) 호출
113         // ...
114         return result;
115     }
116 }

```

RagQueryService는 본 프로젝트에서 실험한 세 가지 실행 모드(SEQUENTIAL, PARALLEL_OLD, PARALLEL_SYNC)를 실제로 구분하여 실행시키는 핵심 서비스이다.

- RagProperties로부터 해석된 executionMode를 사용해 실행 모드별 분기를 수행하고,
- RagTimingRecord를 활용해 각 단계별 시간(guard, retrieval, merge, rerank, context, LLM 등)을 기록한다.
- 마지막으로 [RAG_TIMING] JSON 로그를 한 줄로 출력하여, PowerShell + Python 분석 파이프라인에서 손쉽게 파싱할 수 있도록 하였다.

8.1.4 타이밍 레코드 – RagTimingRecord

```
1 // 파일 경로:
2 // sleepwell-backend/src/main/java/com/sleepwell/sleepwell_backend/rag/model/RagTimingRecord.java
3
4 package com.sleepwell.sleepwell_backend.rag.model;
5
6 // import 생략
7
8 public class RagTimingRecord {
9
10     private final String mode;
11     private final String strategy;
12
13     private long totalMs;
14     private long qExpMs;
15     private long retrievalMs;
16     private long denseMs;
17     private long sparseMs;
18     private long mergeMs;
19     private long rerankMs;
20     private long contextMs;
21     private long llmMs;
22     private long guardrailMs;
23
24     private int threadPoolSize;
25     private String parallelGranularity;
26     private boolean nestedDenseSparse;
27
28     // 생성자, markXxx 메서드, finishTotal, toJson 등 구현
29     // ...
30 }
```

RagTimingRecord는 각 요청에 대한 지연 시간 정보를 모아 JSON으로 직렬화하는 역할을 수행한다.

- TotalMs, QExpMs, RetrievalMs, DenseMs, LlmMs 등 필드를 갖고 있으며,
- 각 단계 시작/종료 시점에 markXxx(...) 메서드가 호출되어 시간을 누적한다.
- 마지막으로 toJson() 메서드를 통해 [RAG_TIMING] { ... } 형식의 로그로 출력된다.

8.2 벤치마크 및 분석 스크립트

8.2.1 서버 실행 스크립트 – run_rag_with_mode.ps1

```
1 # 파일 경로:
2 # sleepwell-backend/tools/run_rag_with_mode.ps1
3
4 param(
5     [Parameter(Mandatory = $true)]
6     [ValidateSet("SEQUENTIAL", "PARALLEL_OLD", "PARALLEL_SYNC")]
7     [string] $Mode,
8
9     [string] $Profile = "rag",
10    [string] $LogPath,
11    [string] $ErrorLogPath
12 )
13
14 Write-Host "Starting RAG backend with RAG_EXECUTION_MODE=$Mode"
15
16 $env:RAG_EXECUTION_MODE = $Mode
17
18 if ($LogPath) {
19     ./gradlew bootRun --args="--spring.profiles.active=$Profile" `
20         2>&1 | Tee-Object -FilePath $LogPath
21 } else {
22     ./gradlew bootRun --args="--spring.profiles.active=$Profile"
23 }
```

이 스크립트는 지정한 모드(-Mode SEQUENTIAL|PARALLEL_OLD|PARALLEL_SYNC)와 프로파일(-Profile rag)로 bootRun을 실행하며, 필요시 콘솔 출력 전체를 로그 파일로 저장한다. 실험에서는 다음과 같이 사용하였다.

- rag_sequential_bootrun.log
- rag_parallel_old_bootrun.log
- rag_async_bootrun.log (PARALLEL_SYNC 모드)

8.2.2 클라이언트 벤치마크 스크립트 - benchmark_rag_client.ps1

```
1 # 파일 경로:
2 # sleepwell-backend/tools/benchmark_rag_client.ps1
3
4 param(
5     [string] $Endpoint = "http://localhost:8080/api/v1/rag/query",
6     [int] $Runs = 30,
7     [string] $ModelLabel = "SEQUENTIAL",
8     [ValidateSet("SEQUENTIAL", "PARALLEL_OLD", "PARALLEL_SYNC")]
9     [string] $ExpectedServerMode,
10    [string] $OutputPath = ".\logs\client_benchmark.csv"
11 )
12
13 # 질문 리스트 예시
14 $queries = @(
15     "cat-1은 일반 수면제와 비교했을 때 어떤 장점이 있나요?",
16     "수면 제한 치료(sat)를 하면 잠드는 시간이나 총 수면 시간이 어떻게 바뀌나요?",
17     "불면증과 불안/우울 증상은 보통 같이 악화되나요?"
18 )
19
20 $results = @()
21
22 for ($i = 0; $i -lt $Runs; $i++) {
23     $q = $queries[$i % $queries.Count]
24     $body = @{ query = $q; topK = 10; namespace = $null } | ConvertTo-Json -Depth 4
25
26     $sw = [System.Diagnostics.Stopwatch]::StartNew()
27     $response = Invoke-WebRequest -Uri $Endpoint -Method POST -Body $body -ContentType "application/json"
28     $sw.Stop()
29
30     $results += [PSCustomObject]@{
31         Index = $i
32         ModelLabel = $ModelLabel
33         ElapsedMs = $sw.ElapsedMilliseconds
34         StatusCode = $response.StatusCode
35     }
36 }
37
38 $results | Export-Csv -NoTypeInformation -Path $OutputPath
39 Write-Host "Benchmark complete. Output: $OutputPath"
```

이 스크립트는 서버 모드를 변경하지 않고, 클라이언트 쪽에서 라벨만 붙여 여러 번 요청을 보낼 때 사용한다. 최종 계측에서는 [RAG_TIMING] 로그 기반 분석으로 전환했지만, 초기 실험에서는 이 스크립트를 통해 평균/최대 응답 시간을 간단히 확인하였다.

8.2.3 로그 → CSV 변환 스크립트 - extract_rag_timing.ps1

```
1 # 파일 경로:
2 # sleepwell-backend/tools/extract_rag_timing.ps1
3
4 param(
5     [Parameter(Mandatory = $true)]
6     [string] $LogPath,
7     [Parameter(Mandatory = $true)]
8     [string] $OutputPath
9 )
10
11 $rows = @()
12
13 Get-Content $LogPath | ForEach-Object {
14     $line = $_
15     if ($line -notmatch "\[RAG_TIMING\]") {
16         return
17     }
18
19     $jsonPart = $line.Split("]")[1].Trim()
20     try {
21         $obj = $jsonPart | ConvertFrom-Json
22         $rows += $obj
23     } catch {
24         Write-Warning "Failed to parse JSON: $jsonPart"
25     }
26 }
27
28 $rows | Export-Csv -NoTypeInformation -Path $OutputPath
29 Write-Host "Extracted timing metrics to $OutputPath"
```

이 스크립트는 bootRun 로그 파일에서 [RAG_TIMING] 라인을 찾아 JSON 부분만 잘라 내고, PowerShell의 ConvertFrom-Json을 이용해 객체로 변환한 뒤 CSV로 저장한다.

실험에서는 다음과 같은 명령으로 사용하였다.

```
1 pwsh .\tools\extract_rag_timing.ps1 `
2   -LogPath ".\logs\rag_sequential_bootrun.log" `
3   -OutputPath ".\logs\rag_sequential_timing.csv"
4
5 pwsh .\tools\extract_rag_timing.ps1 `
6   -LogPath ".\logs\rag_parallel_old_bootrun.log" `
7   -OutputPath ".\logs\rag_parallel_old_timing.csv"
8
9 pwsh .\tools\extract_rag_timing.ps1 `
10  -LogPath ".\logs\rag_async_bootrun.log" `
11  -OutputPath ".\logs\rag_async_timing.csv"
```

8.2.4 분석 스크립트 - analyze_rag_timing.py

```
1 # 파일 경로:
2 # sleepwell-backend/analyze_rag_timing.py
3
4 import argparse
5 import pandas as pd
6 import numpy as np
7
8 def load_dataframe(path: str) -> pd.DataFrame:
9     df = pd.read_csv(path)
10     # mode, totalMs 필드가 있는지 확인
11     if "mode" not in df.columns or "totalMs" not in df.columns:
12         raise ValueError("CSV must contain at least 'mode' and 'totalMs' columns")
13     return df
14
15 def summarize_total_latency(df: pd.DataFrame) -> pd.DataFrame:
16     def p95(x):
17         return np.percentile(x, 95)
18     grouped = df.groupby("mode")["totalMs"]
19     summary = grouped.agg(["count", "mean", "median", "min", "max"])
20     summary["p95"] = grouped.apply(p95)
21     return summary
22
23 def summarize_stage_latency(df: pd.DataFrame) -> pd.DataFrame:
24     stage_cols = [
25         "denseMs", "sparseMs", "mergeMs", "rerankMs",
26         "contextMs", "llmMs", "qExpMs", "retrievalMs", "guardrailMs"
27     ]
28     stage_cols = [c for c in stage_cols if c in df.columns]
29     if not stage_cols:
30         return pd.DataFrame()
31
32     def p95(x):
33         return np.percentile(x, 95)
34
35     metrics = {}
36     for col in stage_cols:
37         grouped = df.groupby("mode")[col]
38         metrics[f"{col}_mean"] = grouped.mean()
39         metrics[f"{col}_median"] = grouped.median()
40         metrics[f"{col}_p95"] = grouped.apply(p95)
41
42     stage_df = pd.DataFrame(metrics)
43     return stage_df
44
45 def main():
46     parser = argparse.ArgumentParser()
47     parser.add_argument("--csv", default="rag_timing_metrics.csv")
48     args = parser.parse_args()
49
50     df = load_dataframe(args.csv)
51
52     total_summary = summarize_total_latency(df)
53     print("=== Total Latency Summary by Mode (ms) ===")
54     print(total_summary)
55
56     stage_summary = summarize_stage_latency(df)
57     if not stage_summary.empty:
58         print("\n=== Stage-wise Latency Summary by Mode (ms) ===")
59         print(stage_summary)
60
61 if __name__ == "__main__":
62     main()
```

이 스크립트는

1. rag_timing_metrics.csv 파일을 읽어들이고,
2. 모드별로 totalMs에 대해 count, mean, median, min, max, p95를 계산하고,
3. 존재하는 경우 denseMs, retrievalMs, llmMs 등 단계별 시간에 대해서도 mean/median/p95를 계산하여 출력한다.

보고서 본문 5장에서 제시한 평균/ p95 응답 시간 표와 단계별 평균 시간 표는 모두 이 스크립트의 결과를 기반으로 작성되었다.